Prelucrarea Imaginilor

Prof. Dr. Călin-Adrian POPA

Cursul 7

10 Noiembrie 2025

7 Semantic segmentation

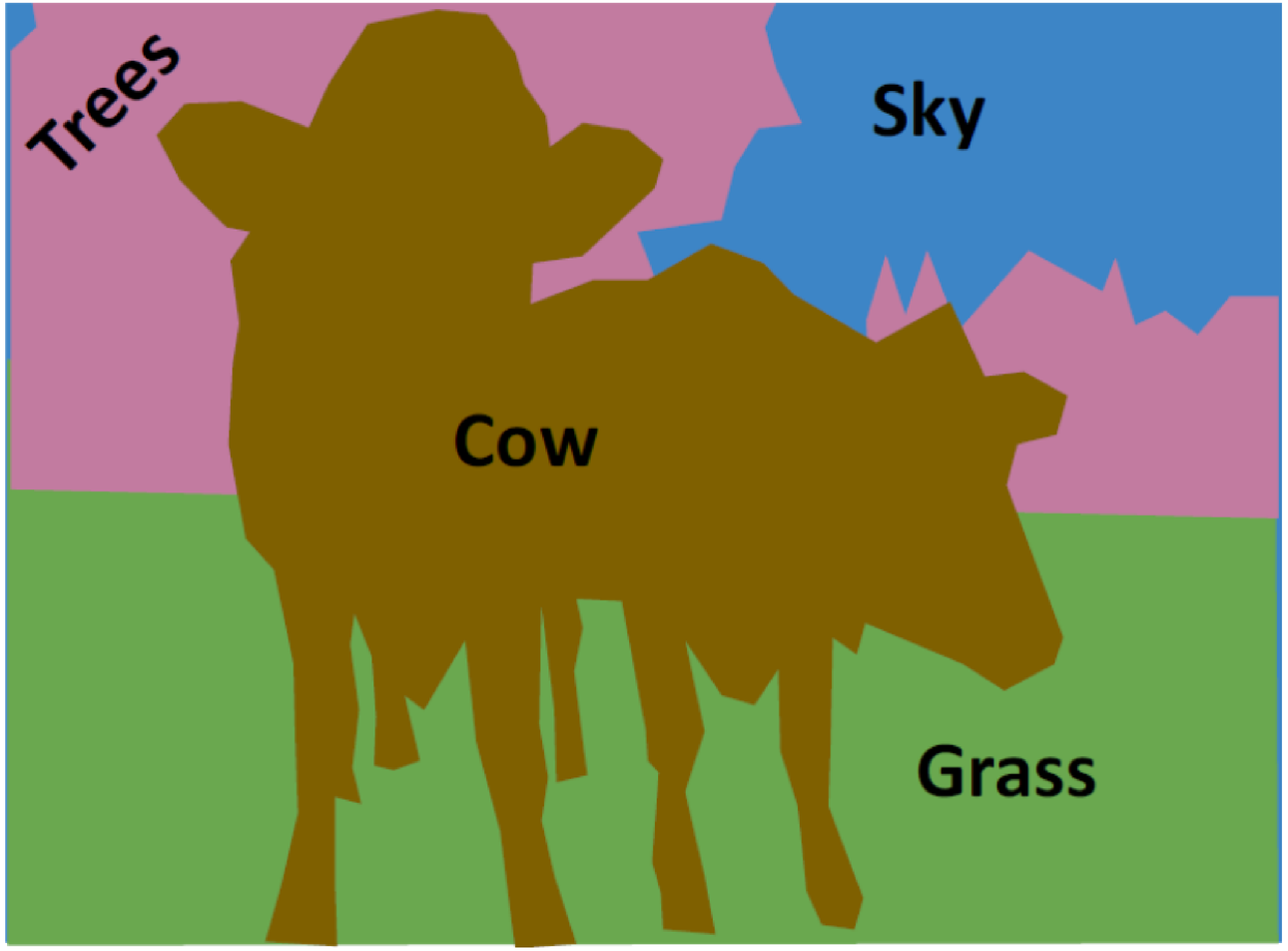
- when discussing object detection tasks in Chapter 6, rectangular bounding boxes are used to label and predict objects in images



7 Semantic segmentation

- this chapter will discuss the problem of *semantic segmentation*, which focuses on how to divide an image into regions belonging to different semantic classes
- unlike object detection, semantic segmentation recognizes and understands the content of images at pixel level: its labeling and prediction of semantic regions are at pixel level
- compared with the case of object detection, the pixel-level borders labeled in semantic segmentation are obviously more fine-grained

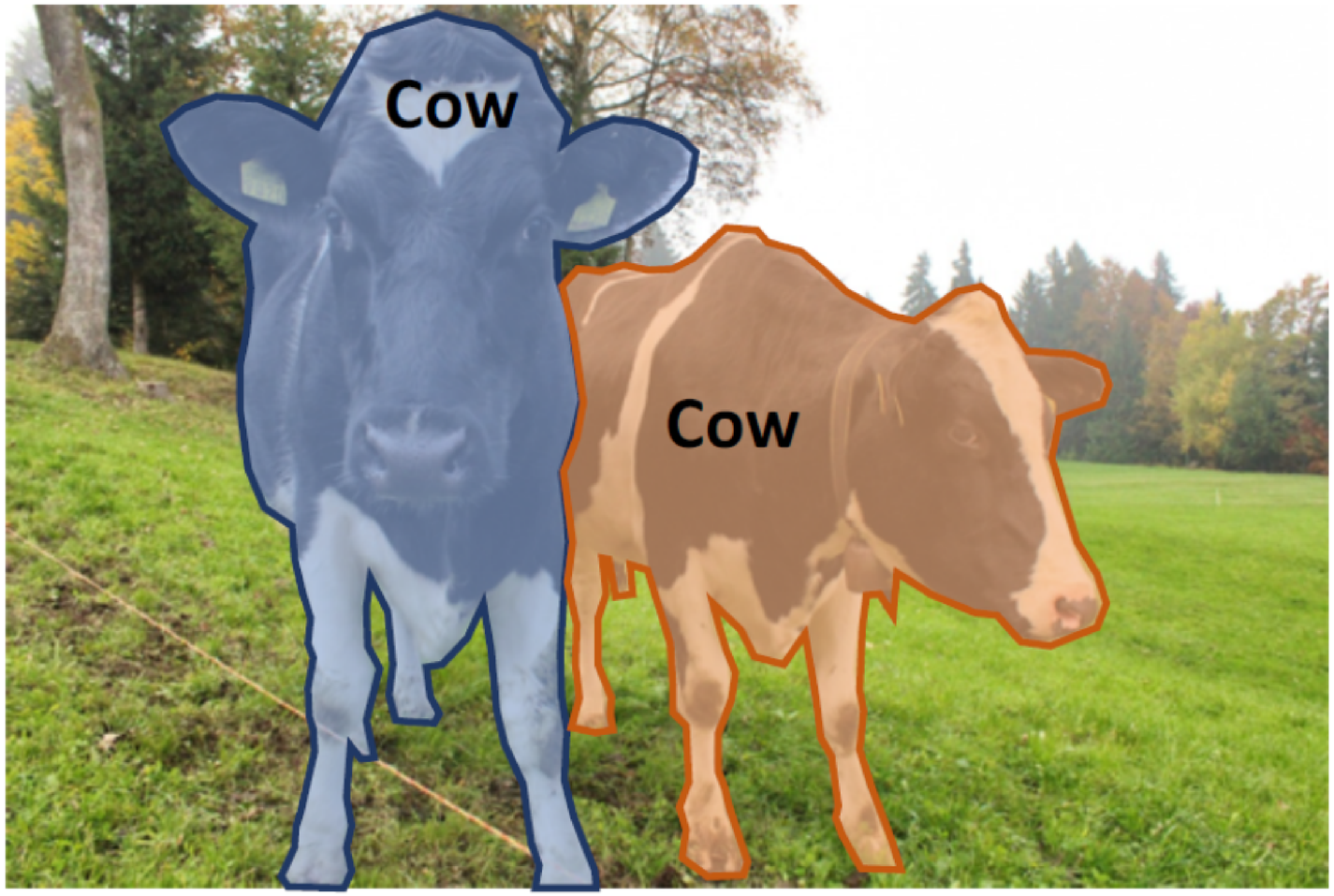
7 Semantic segmentation



7 Semantic segmentation

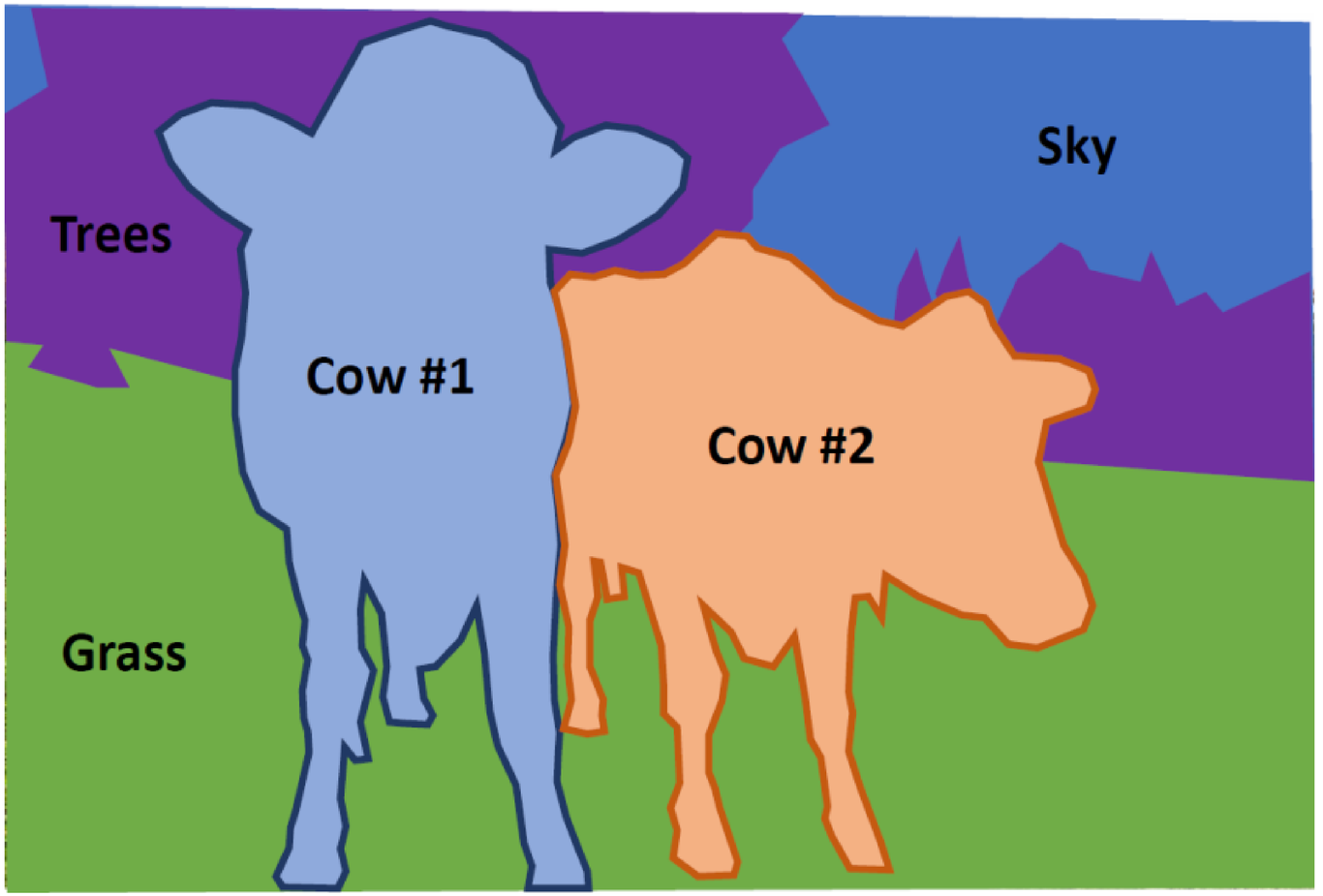
- there are also two important tasks in the field of computer vision that are similar to semantic segmentation, namely instance segmentation and panoptic segmentation
- *instance segmentation* is also called *simultaneous detection and segmentation*
- it studies how to recognize the pixel-level regions of each object instance in an image
- different from semantic segmentation, instance segmentation needs to distinguish not only semantics, but also different object instances
- for example, if there are two cows in the image, instance segmentation needs to distinguish which of the two cows a pixel belongs to

7 Semantic segmentation



7 Semantic segmentation

- *panoptic segmentation* gives labels to uncountable objects called "stuff" (sky, grass, etc.), similar to semantic segmentation, and differentiates between pixels coming from different instances of the same class (countable objects) called "things" (cows, etc.), similar to instance segmentation

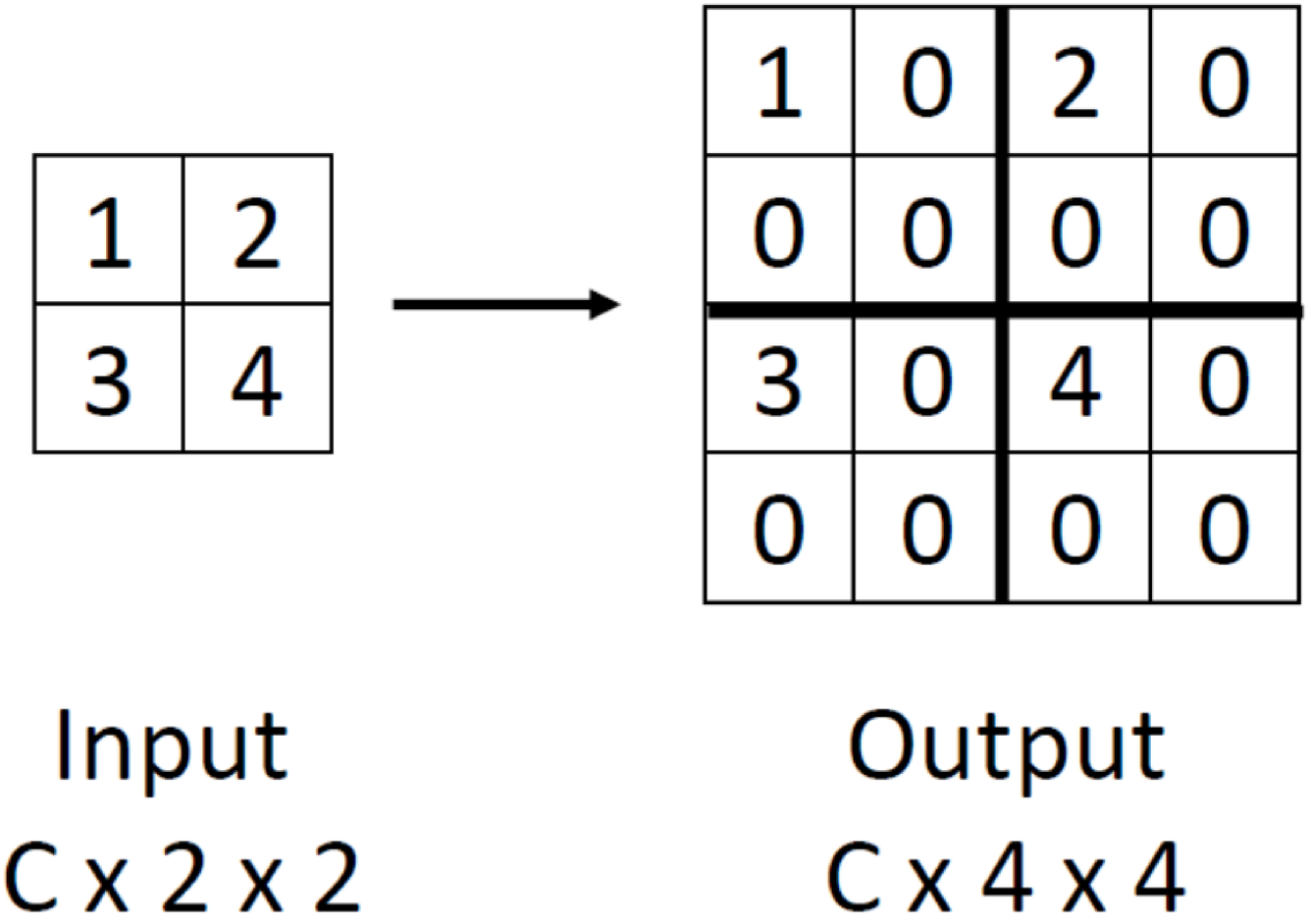


7.1 Upsampling

- the CNN layers we have seen so far, such as convolutional layers (Section 3.2) and pooling layers (Section 3.5), typically reduce (downsample) the spatial dimensions (height and width) of the input, or keep them unchanged
- in semantic segmentation that classifies at pixel-level, it will be convenient if the spatial dimensions of the input and output are the same
- for example, the channel dimension at one output pixel can hold the classification results for the input pixel at the same spatial position
- to achieve this, especially after the spatial dimensions are reduced by CNN layers, we can use other types of CNN layers that can increase (upsample) the spatial dimensions of intermediate feature maps

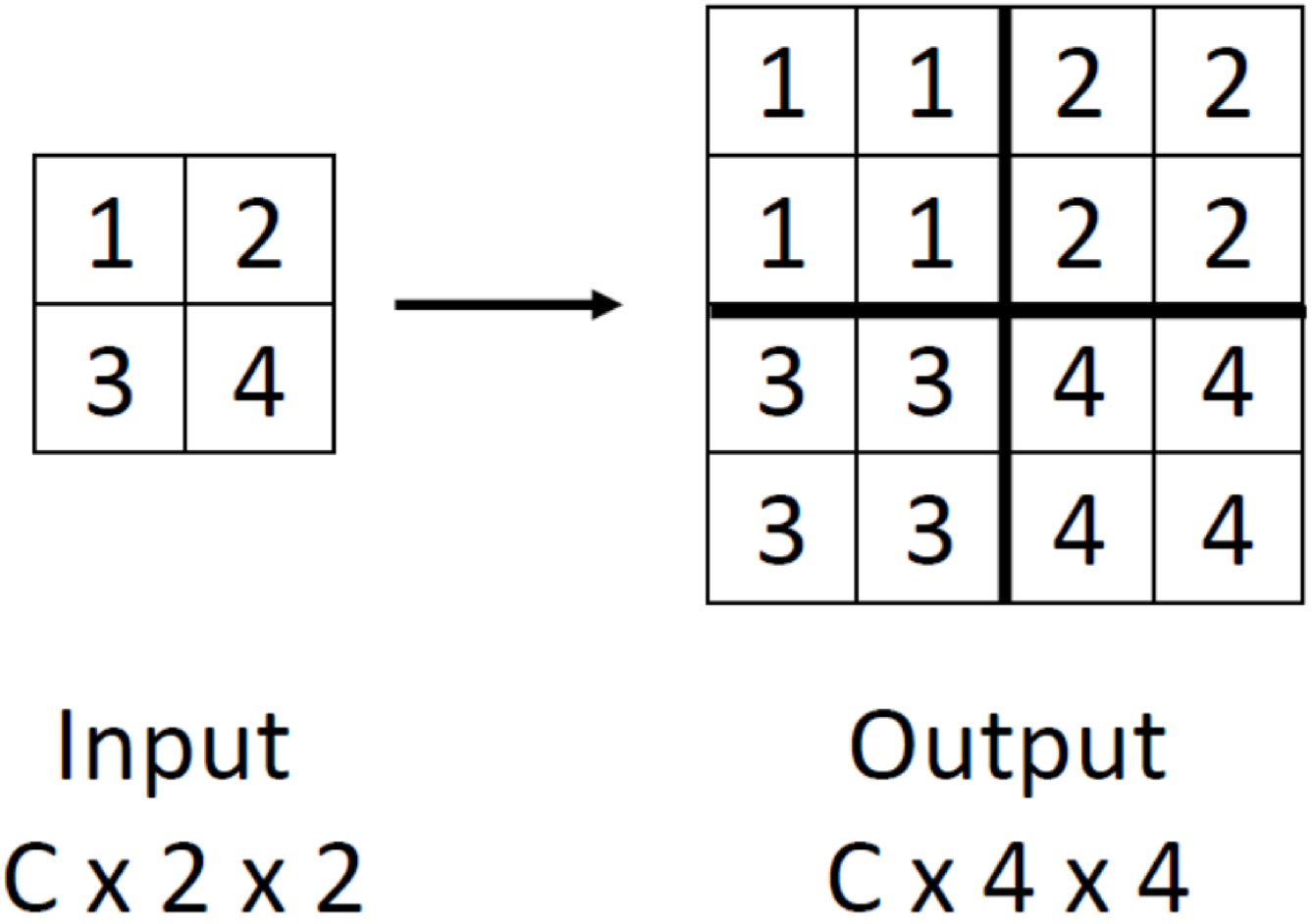
7.1 Upsampling

- *unpooling – bed of nails*



7.1 Upsampling

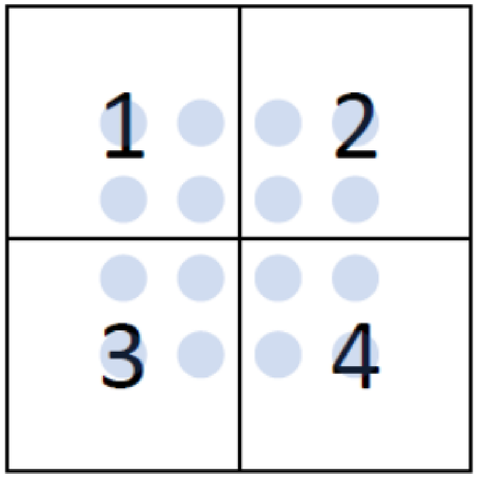
- *unpooling – nearest neighbor*



7.1 Upsampling

- *bilinear interpolation* uses four closest neighbors of (x, y) to construct linear approximations

$$f_{x,y} = \sum_{\substack{i\in\{\lfloor x\rfloor-1,\ldots,\lceil x\rceil+1\}\\ j\in\{\lfloor y\rfloor-1,\ldots,\lceil y\rceil+1\}}} f_{i,j}\max(0,1-|x-i|)\max(0,1-|y-j|)$$





1.00	1.25	1.75	2.00
1.50	1.75	2.25	2.50
2.50	2.75	3.25	3.50
3.00	3.25	3.75	4.00

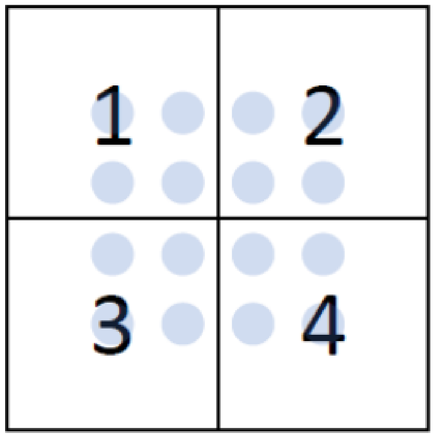
Input: C x 2 x 2

Output: C x 4 x 4

7.1 Upsampling

- *bicubic interpolation* uses sixteen closest neighbors of (x, y) to construct cubic approximations

$$f_{x,y} = \sum_{\substack{i\in\{\lfloor x\rfloor-3,\ldots,\lceil x\rceil+3\}\\ j\in\{\lfloor y\rfloor-3,\ldots,\lceil y\rceil+3\}}} f_{i,j} \max(0, 3-|x-i|) \max(0, 3-|y-j|)$$



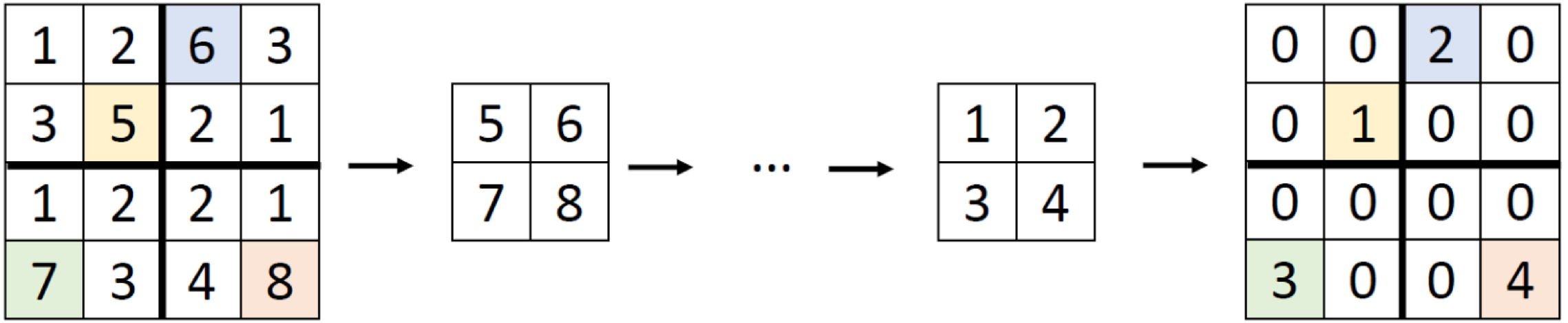
0.68	1.02	1.56	1.89
1.35	1.68	2.23	2.56
2.44	2.77	3.32	3.65
3.11	3.44	3.98	4.32

Input: C x 2 x 2

Output: C x 4 x 4

7.1 Upsampling

- for *max-unpooling*, we remember which position gave the maximum in max-pooling, and at max-unpooling we place the values into the remembered positions
- each downsampling layer is paired with an upsampling layer



7.1 Upsampling

- now, we will introduce *transposed convolution*, which is also called *fractionally-strided convolution*, for reversing downsampling operations by the convolution
- ignoring channels for now, let us begin with the basic transposed convolution operation with stride of 1 and no padding
- suppose that we are given a $n_h \times n_w$ input tensor and a $k_h \times k_w$ kernel
- sliding the kernel window with stride of 1 for n_w times in each row and n_h times in each column, yields a total of $n_h n_w$ intermediate results
- each intermediate result is a $(n_h + k_h - 1) \times (n_w + k_w - 1)$ tensor that is initialized with zeros

7.1 Upsampling

- to compute each intermediate tensor, each element in the input tensor is multiplied by the kernel, so that the resulting $k_h \times k_w$ tensor replaces a portion in each intermediate tensor
- note that the position of the replaced portion in each intermediate tensor corresponds to the position of the element in the input tensor used for the computation
- in the end, all the intermediate results are summed over to produce the output
- as an example, Figure 1 illustrates how transposed convolution with a 2×2 kernel is computed for a 2×2 input tensor

7.1 Upsampling

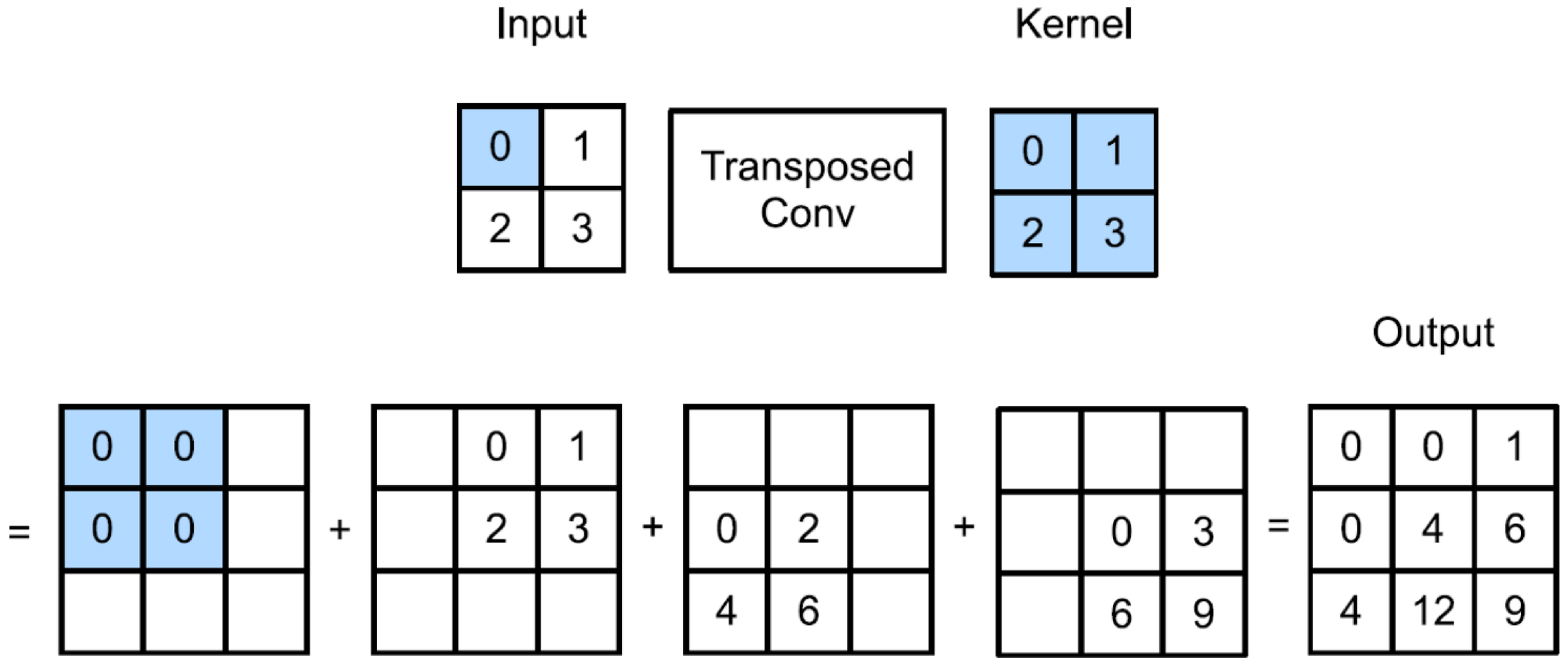


Figure 1: Transposed convolution with a 2×2 kernel. The shaded portions are a portion of an intermediate tensor, as well as the input and kernel tensor elements used for the computation.

7.1 Upsampling

- in contrast to the regular convolution (in Section 3.2) that *reduces* input elements via the kernel, the transposed convolution *broadcasts* input elements via the kernel, thereby producing an output that is larger than the input
- different from the regular convolution, where padding is applied to input, it is applied to output in the transposed convolution
- for example, when specifying the padding number on either side of the height and width as 1, the first and last rows and columns will be removed from the transposed convolution output
- in the transposed convolution, strides are specified for intermediate results (thus output), not for input
- using the same input and kernel tensors from Figure 1, changing the stride from 1 to 2 increases both the height and width of intermediate tensors, hence the output tensor in Figure 2

7.1 Upsampling

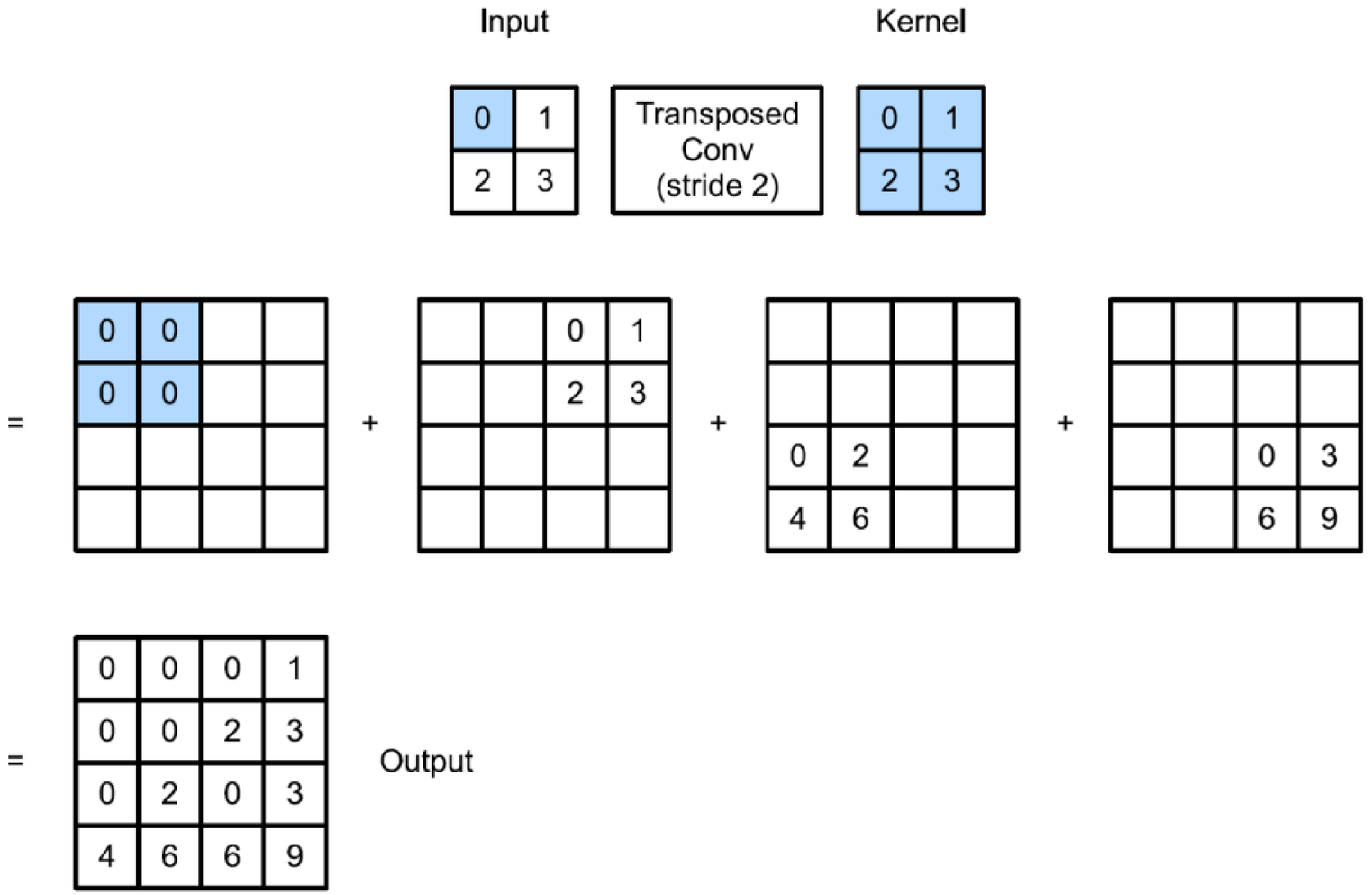


Figure 2: Transposed convolution with a 2×2 kernel with stride of 2. The shaded portions are a portion of an intermediate tensor, as well as the input and kernel tensor elements used for the computation.

7.1 Upsampling

- for multiple input and output channels, the transposed convolution works in the same way as the regular convolution
- suppose that the input has c_i channels, and that the transposed convolution assigns a $k_h \times k_w$ kernel tensor to each input channel
- when multiple output channels are specified, we will have a $c_i \times k_h \times k_w$ kernel for each output channel
- all in all, if we feed X into a convolutional layer f to output $\mathrm{Y} = f(\mathrm{X})$ and create a transposed convolutional layer g with the same hyperparameters as f, except for the number of output channels being the number of channels in X, then $g(\mathrm{Y})$ will have the same shape as X
- in general, we can see that for stride s, padding $s/2$ (assuming $s/2$ is an integer), and the height and width of the kernel $2s$, the transposed convolution will increase the height and width of the input by a factor of s

7.2 Fully convolutional networks

- as previously discussed, semantic segmentation classifies images at pixel level
- a *fully convolutional network* (*FCN*) uses a convolutional neural network to transform image pixels to pixel classes
- unlike the CNNs that we encountered earlier for image classification or object detection, a fully convolutional network transforms the height and width of intermediate feature maps back to those of the input image: this is achieved by the transposed convolutional layer introduced in Section 7.1
- as a result, the classification output and the input image have a one-to-one correspondence at pixel level: the channel dimension at any output pixel holds the classification results for the input pixel at the same spatial position

7.2 Fully convolutional networks

- here, we describe the basic design of the fully convolutional network model
- as shown in Figure 3, this model first uses a CNN to extract image features, then transforms the number of channels into the number of classes via a 1×1 convolutional layer, and finally transforms the height and width of the feature maps to those of the input image via the transposed convolution introduced in Section 7.1
- as a result, the model output has the same height and width as the input image, where the output channel contains the predicted classes for the input pixel at the same spatial position
- upsampling is done by a transposed convolution initialized with bilinear interpolation

7.2 Fully convolutional networks

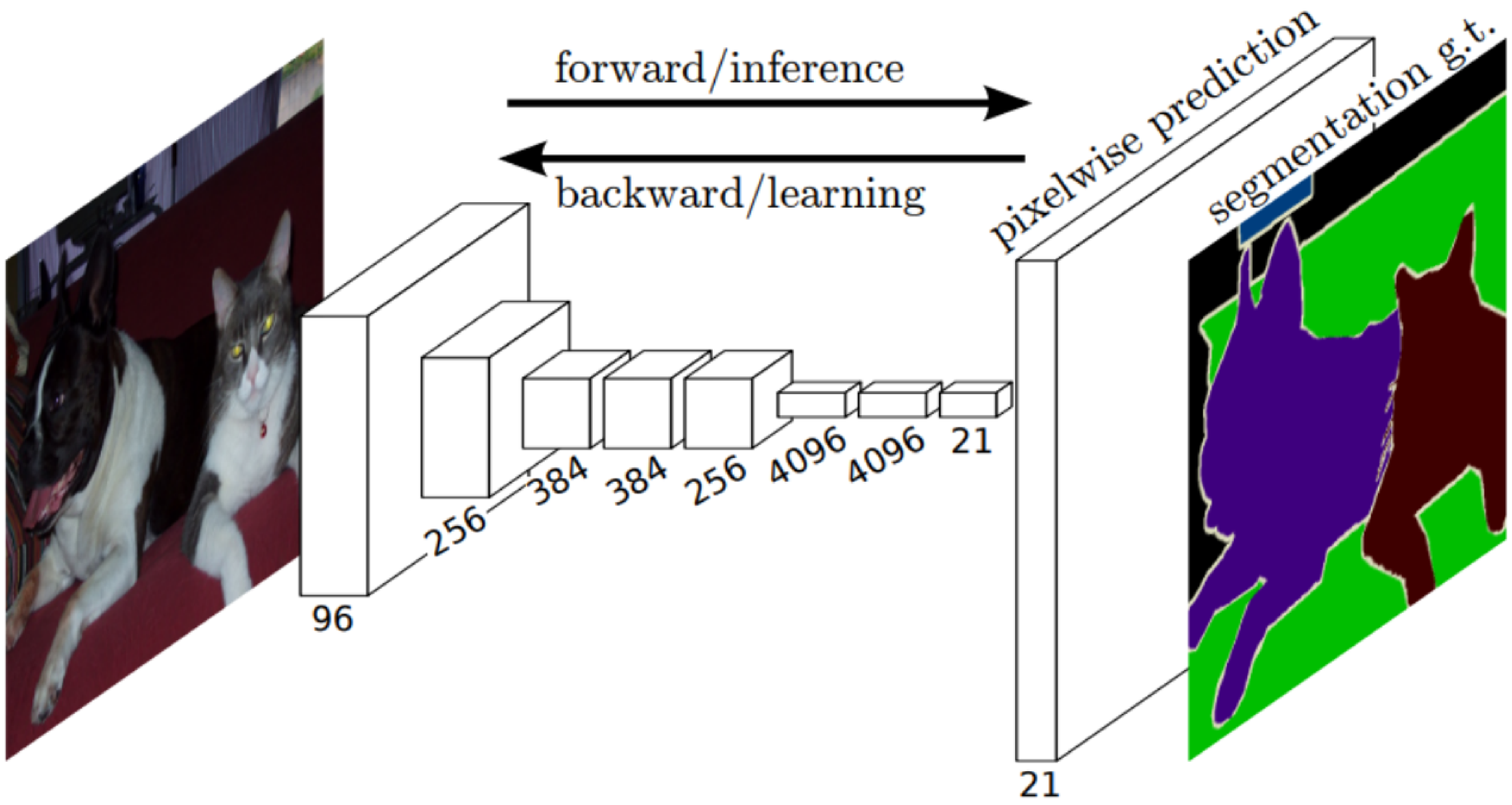


Figure 3: Fully convolutional networks can efficiently learn to make dense predictions for per-pixel tasks like semantic segmentation.

7.2 Fully convolutional networks

- fully connected layers can also be viewed as convolutions with kernels that cover their entire input regions
- doing so casts them into fully convolutional networks that take input of any size and output classification maps
- this transformation is illustrated in Figure 4

7.2 Fully convolutional networks

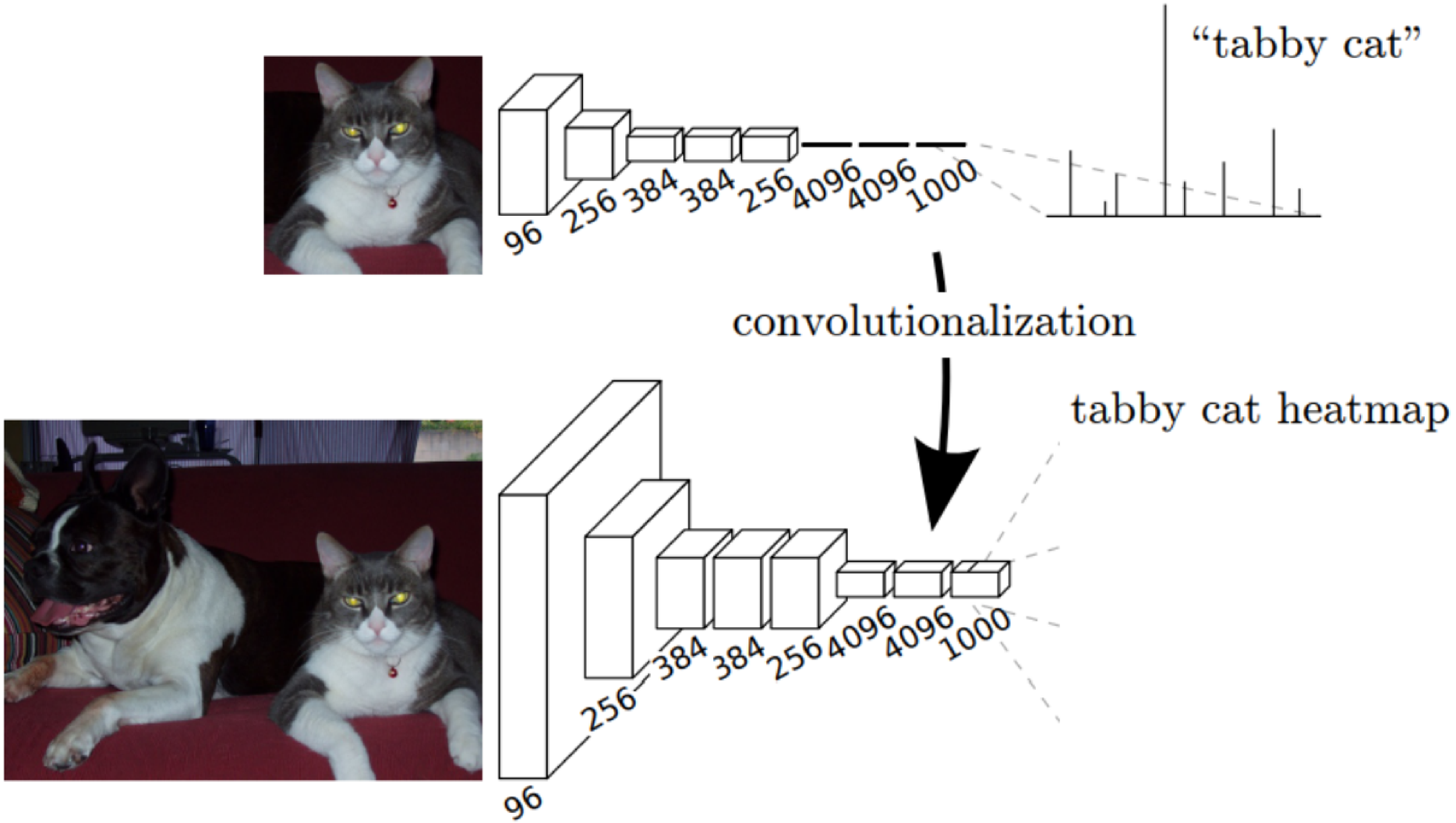


Figure 4: Transforming fully connected layers into convolution layers enables a classification network to output a heatmap.

7.2 Fully convolutional networks

- hierarchical training is used, where the network is initially trained only based on high-level features and then fine-tuned based on middle and low-level features
- this is important because it allows the network to also learn the middle and low-level details of the image, in addition to high-level ones
- initially, predict the segmentation mask from high-level features (FCN-32s)
- then, predict the segmentation mask from middle-level features (FCN-16s)
- finally, predict the segmentation mask from low-level features (FCN-8s)
- the whole process is illustrated in Figure 5
- an example showing the effectiveness of this approach is shown in Figure 6

7.2 Fully convolutional networks

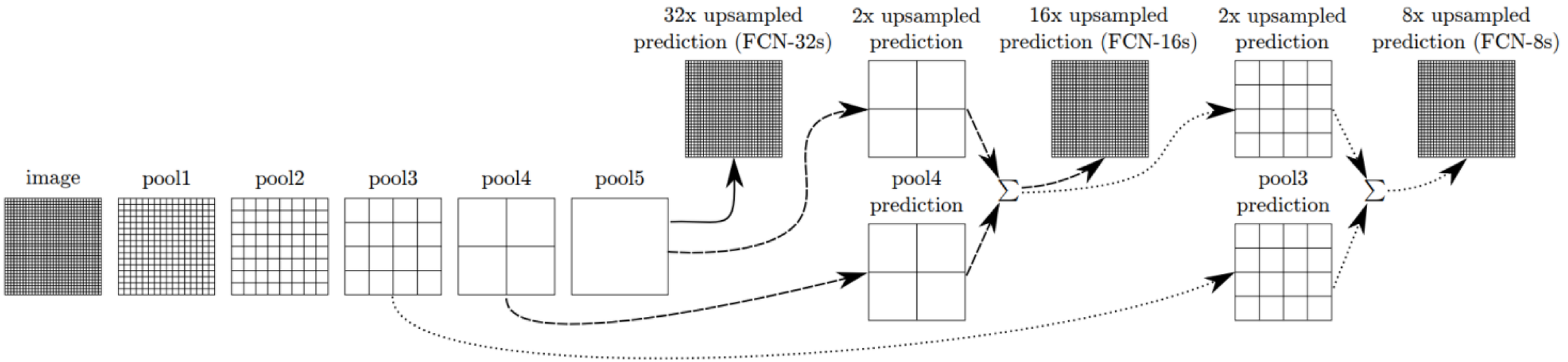


Figure 5: The FCNs learn to combine coarse, high-layer information with fine, low-layer information. Layers are shown as grids that reveal relative spatial coarseness. Only pooling and prediction layers are shown; intermediate convolution layers (including the converted fully connected layers) are omitted. Solid line (FCN-32s): the single-stream network, upsamples stride 32 predictions back to pixels in a single step. Dashed line (FCN-16s): combining predictions from both the final layer and the pool4 layer, at stride 16, lets the network predict finer details, while retaining high-level semantic information. Dotted line (FCN-8s): additional predictions from pool3, at stride 8, provide further precision.

7.2 Fully convolutional networks

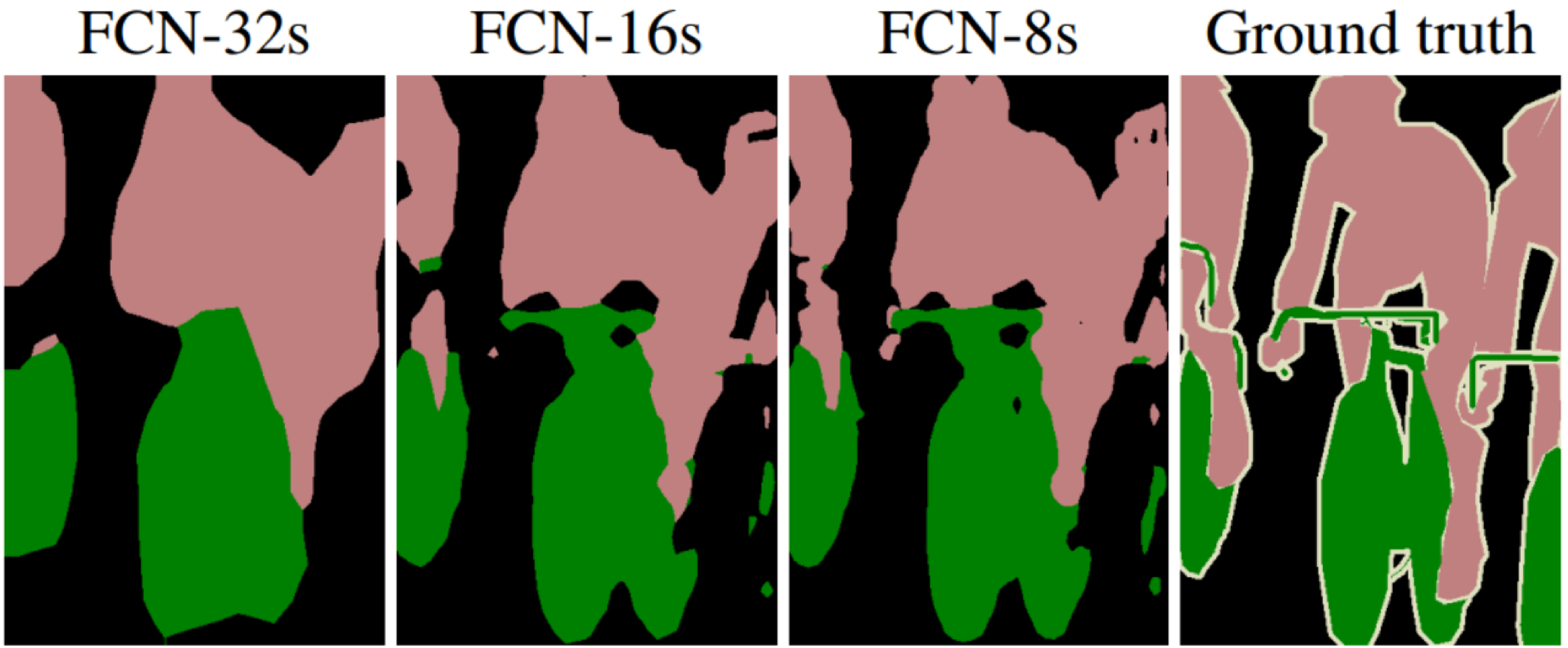


Figure 6: Refining FCNs by fusing information from layers with different strides improves segmentation detail. The first three images show the output from the 32, 16, and 8 pixel stride networks.

7.2 Fully convolutional networks

- now, we can discuss the training of the fully convolutional network
- the loss function and accuracy calculation are not essentially different from those in image classification
- because we use the output channel of the transposed convolutional layer to predict the class for each pixel, the channel dimension is specified in the loss calculation
- here, we use the softmax operation to predict the class, and we have 21 output channels because the Pascal VOC2012 dataset has 20 classes, and we add the background class
- the loss function is a per-pixel cross-entropy loss
- in addition, the accuracy is calculated based on the correctness of the predicted class, for all the pixels
- when predicting, we need to standardize the input image in each channel and transform the image into the four-dimensional input format required by the CNN
- to visualize the predicted class of each pixel, we map the predicted class back to its label color in the dataset

7.3 SegNet

- the idea of the SegNet model is to perform step-wise upsampling
- SegNet has an encoder network and a corresponding decoder network, followed by a final pixelwise classification layer
- this architecture is illustrated in Figure 7
- the encoder network consists of 13 convolutional layers which correspond to the first 13 convolutional layers in the VGG-16 network
- we can therefore initialize the training process from weights trained for classification on large datasets, e.g., ImageNet
- the fully connected layers are discarded in favor of retaining higher-resolution feature maps at the deepest encoder output
- each encoder layer has a corresponding decoder layer and hence the decoder network has 13 layers
- the final decoder output is fed to a multi-class softmax classifier to produce class probabilities for each pixel independently

7.3 SegNet

- each encoder in the encoder network performs convolution to produce a set of feature maps
- these are then batch normalized, and then an elementwise ReLU is applied
- following that, max-pooling with a 2×2 window and stride 2 is performed and the resulting output is sub-sampled by a factor of 2
- while several layers of max-pooling can achieve more translation invariance for robust classification, correspondingly there is a loss of spatial resolution of the feature maps
- the increasingly lossy (boundary detail) image representation is not beneficial for segmentation, where boundary delineation is vital
- therefore, it is necessary to capture and store boundary information in the encoder feature maps before sub-sampling is performed
- because all the encoder feature maps (after subsampling) cannot be stored, a more efficient way to store this information involves storing only the max-pooling indices, i.e., the locations of the maximum feature value in each pooling window is memorized for each encoder feature map

7.3 SegNet

- the appropriate decoder in the decoder network upsamples its input feature maps using the memorized encoder max-pooling indices from the corresponding encoder feature maps, i.e., it performs max-unpooling
- this step produces sparse feature maps
- these feature maps are then processed using trainable convolutional layers to produce dense feature maps, i.e., the goal of the convolutional layers is to refine the upsampling
- a batch normalization step is then applied to each of these maps
- note that the decoder corresponding to the first encoder (closest to the input image) produces a multi-channel feature map, although its encoder input has 3 channels (RGB)
- this is unlike the other decoders in the network which produce feature maps with the same number of size and channels as their encoder inputs
- the high-dimensional feature representation at the output of the final decoder is fed to a trainable softmax classifier

7.3 SegNet

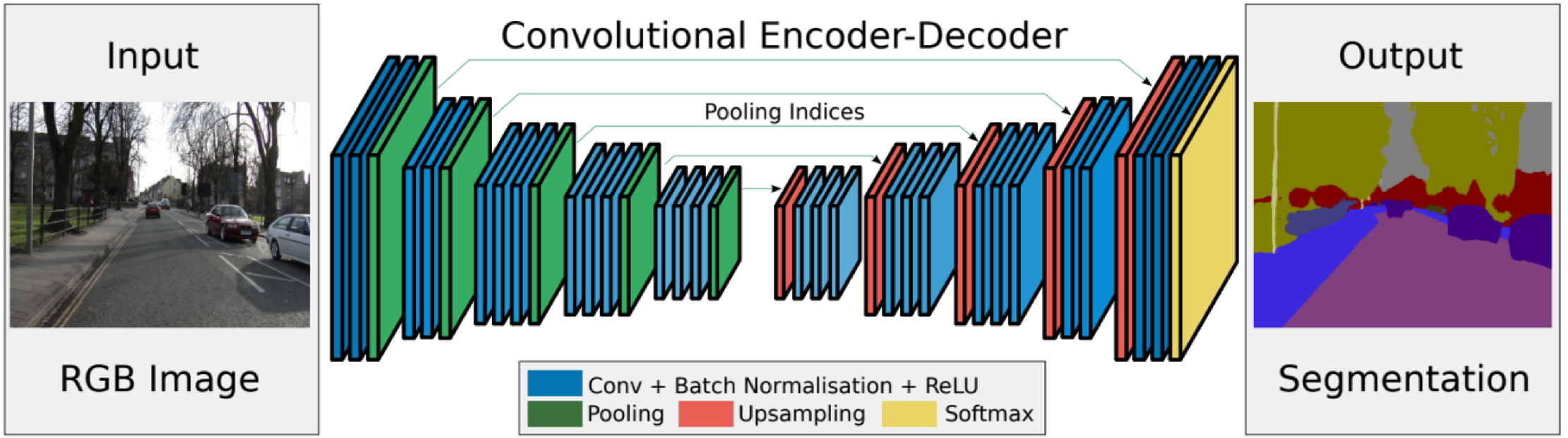


Figure 7: An illustration of the SegNet architecture. There are no fully connected layers and hence it is only convolutional. The decoder upsamples its input using the transferred pooling indices from the encoder to produce a sparse feature map. It then performs convolution with trainable filters to densify the feature map. The final decoder output feature maps are fed to a softmax classifier for pixelwise classification.

7.4 U-Net

- the main idea of FCNs is to supplement a usual contracting network by successive layers, where pooling operators are replaced by upsampling operators
- hence, these layers increase the resolution of the output
- in order to localize, high-resolution features from the contracting path are combined with the upsampled output
- a successive convolution layer can then learn to assemble a more precise output based on this information
- one important modification in the U-Net architecture is that in the upsampling part we have also a large number of feature channels, which allow the network to propagate context information to higher-resolution layers
- as a consequence, the expansive path is more or less symmetric to the contracting path, and yields a U-shaped architecture
- the network does not have any fully connected layers and only uses the valid part of each convolution, i.e., the segmentation map only contains the pixels for which the full context is available in the input image
- this strategy allows the seamless segmentation of arbitrarily large images

7.4 U-Net

- the network architecture is illustrated in Figure 8
- it consists of a contracting path (left side) and an expansive path (right side)
- the contracting path follows the typical architecture of a convolutional network
- it consists of the repeated application of two 3×3 convolutions (unpadded convolutions), each followed by a rectified linear unit (ReLU) and a 2×2 max-pooling operation with stride 2 for downsampling
- at each downsampling step, we double the number of feature channels
- every step in the expansive path consists of an upsampling of the feature map followed by a 2×2 transposed convolution ("up-convolution") that halves the number of feature channels, a concatenation with the correspondingly cropped feature map from the contracting path (skip connection, similar to ResNet), and two 3×3 convolutions, each followed by a ReLU
- the cropping is necessary due to the loss of border pixels in every convolution
- at the final layer a 1×1 convolution is used to map each 64-component feature vector to the desired number of classes
- in total, the network has 23 convolutional layers

7.4 U-Net

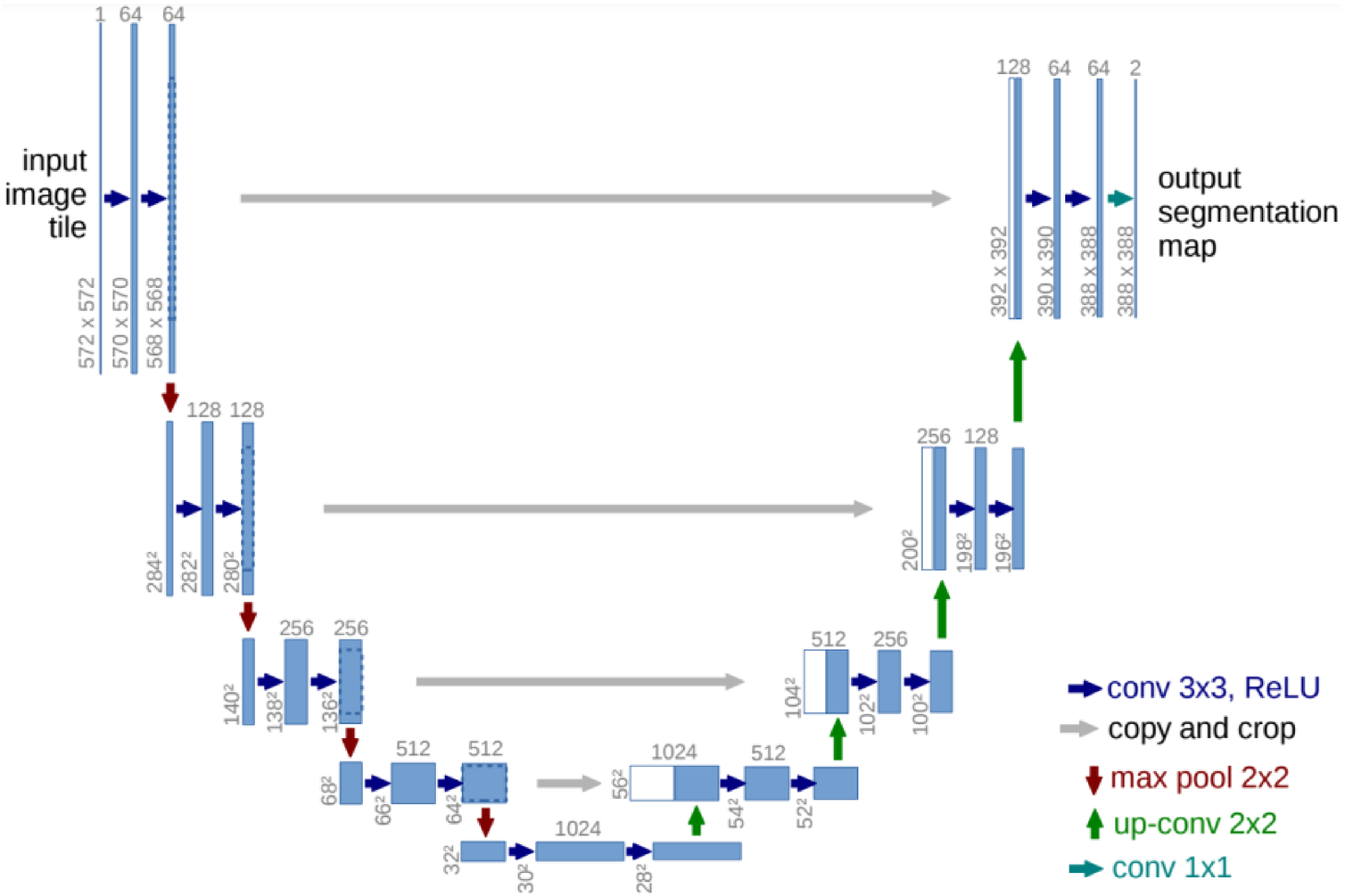


Figure 8: U-Net architecture. Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The *x*-*y* size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

7.5 Dilated convolution

- the repeated combination of max-pooling and striding at consecutive layers of these networks reduces significantly the spatial resolution of the resulting feature maps, typically by a factor of 32 across each direction
- a partial remedy is to use transposed convolution layers, which however requires additional memory and time
- instead, *dilated (atrous) convolution*, originally developed for the efficient computation of the wavelet transform in signal processing, can be used
- this algorithm allows us to compute the responses of any layer at any desirable resolution
- considering one-dimensional signals first, the output y_i of dilated convolution of a 1D input signal x_i with a filter w_k of length K is defined as:

$$y_i = \sum_{k=1}^{K} x_{i+r \cdot k} w_k.$$

- the rate parameter r corresponds to the stride with which we sample the input signal
- standard convolution is a special case for rate $r = 1$; see Figure 9 for illustration

7.5 Dilated convolution

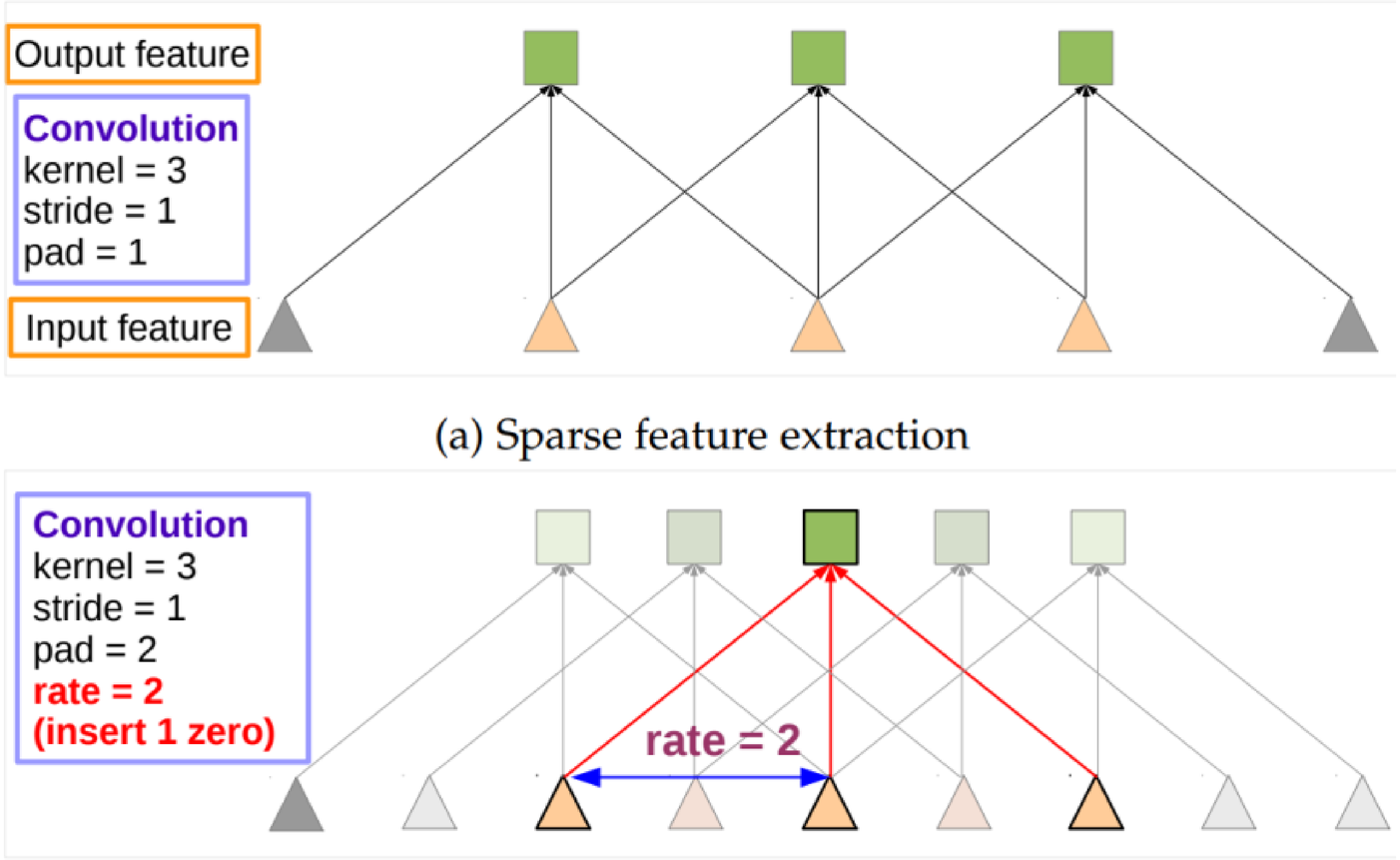


Figure 9: Illustration of dilated convolution in 1D. (a) Sparse feature extraction with standard convolution on a low-resolution input feature map. (b) Dense feature extraction with dilated convolution with rate $r = 2$, applied on a high-resolution input feature map.

7.5 Dilated convolution

- we illustrate the algorithm's operation in 2D through a simple example in Figure 10: given an image, we assume that we first have a downsampling operation that reduces the resolution by a factor of 2, and then perform a convolution with a kernel
- if we implant the resulting feature map in the original image coordinates, we realize that we have obtained responses at only $1/4$ of the image positions
- instead, we can compute responses at all image positions if we perform a convolution of the full-resolution image with a filter "with holes", in which we upsample the original filter by a factor of 2, and introduce zeros in between filter values
- although the effective filter size increases, we only need to take into account the non-zero filter values, hence both the number of filter parameters and the number of operations per position stay constant
- the resulting scheme allows us to easily and explicitly control the spatial resolution of neural network feature maps

7.5 Dilated convolution

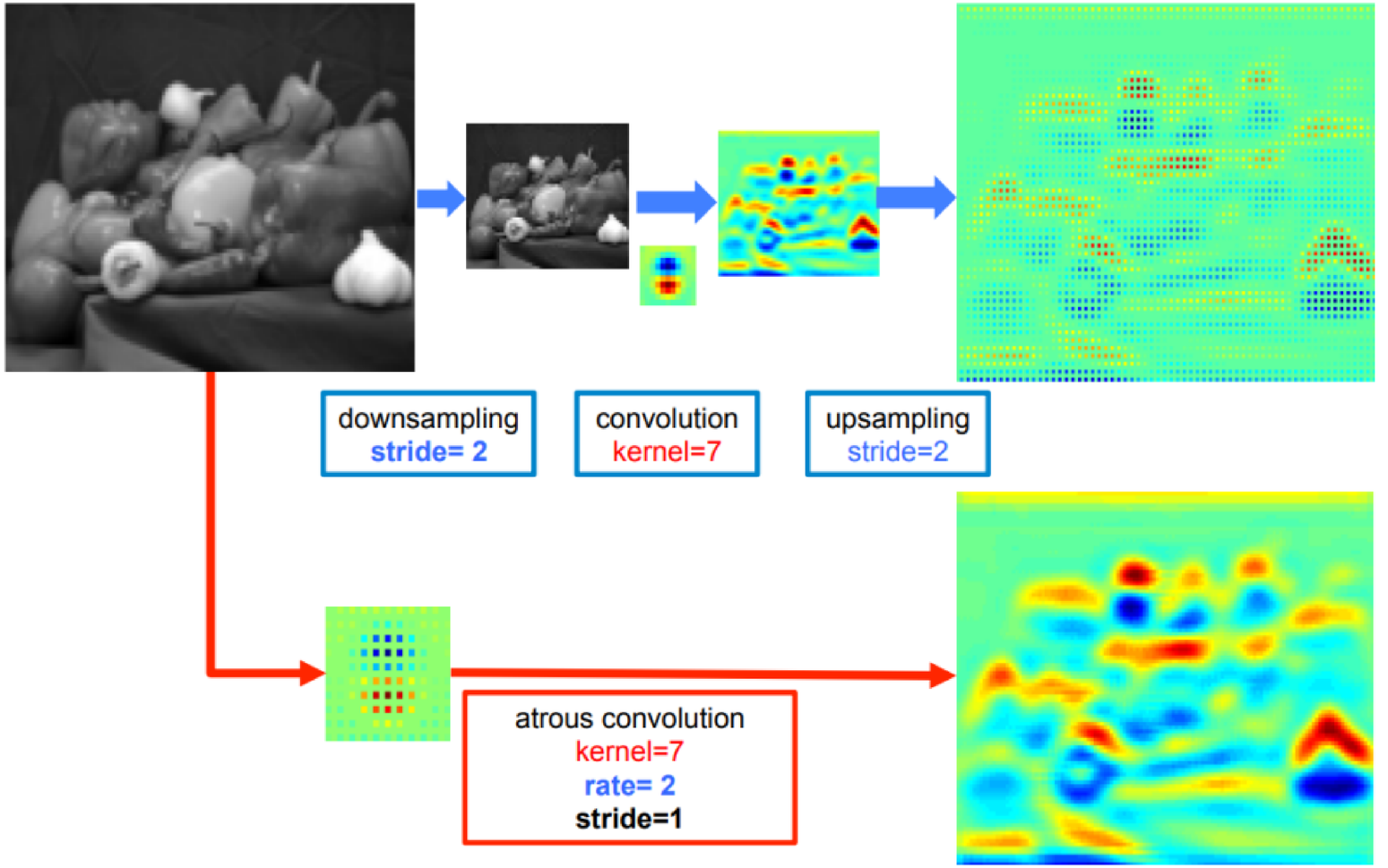


Figure 10: Illustration of dilated convolution in 2D. Top row: sparse feature extraction with standard convolution on a low-resolution input feature map. Bottom row: dense feature extraction with dilated convolution with rate $r = 2$, applied on a high-resolution input feature map.

7.5 Dilated convolution

- in the context of CNNs, we can use dilated convolution in a chain of layers, effectively allowing us to compute the final feature maps at an arbitrarily high resolution
- for example, in order to double the spatial density of computed feature maps in the VGG-16 or ResNet-101 networks, we find the last pooling or convolutional layer that decreases resolution, set its stride to 1 to avoid signal decimation, and replace all subsequent convolutional layers with atrous convolutional layers having rate $r = 2$
- pushing this approach all the way through the network could allow us to compute feature maps at the original image resolution, but this ends up being too costly
- a hybrid approach that has a good efficiency/accuracy trade-off, uses dilated convolution to increase by a factor of 4 the density of computed feature maps, followed by fast bilinear interpolation by an additional factor of 8 to recover feature maps at the original image resolution
- bilinear interpolation is sufficient in this setting because the class score maps (corresponding to probabilities) are quite smooth
- unlike the transposed convolution approach, this approach converts image classification networks into dense feature extractors without requiring learning any extra parameters, leading to faster training in practice

7.5 Dilated convolution

- dilated convolution also allows us to arbitrarily enlarge the receptive fields of filters at any layer
- CNNs typically employ spatially small convolution kernels (typically 3×3) in order to keep both computation and number of parameters contained
- dilated convolution with rate r introduces $r - 1$ zeros between consecutive filter values, effectively enlarging the kernel size of a $k \times k$ filter to $k_e = k + (k - 1)(r - 1)$ without increasing the number of parameters or the amount of computation
- it thus offers an efficient mechanism to control the receptive field and finds the best trade-off between accurate localization (small receptive field) and context assimilation (large receptive field), as shown in Figure 11
- in general, when the dilation for the height is r_h and the dilation for the width is r_w, the output shape is:

$$\lfloor (n_h - r_h(k_h - 1) - 1 + p_h + s_h)/s_h \rfloor \times \lfloor (n_w - r_w(k_w - 1) - 1 + p_w + s_w)/s_w \rfloor.$$

7.5 Dilated convolution

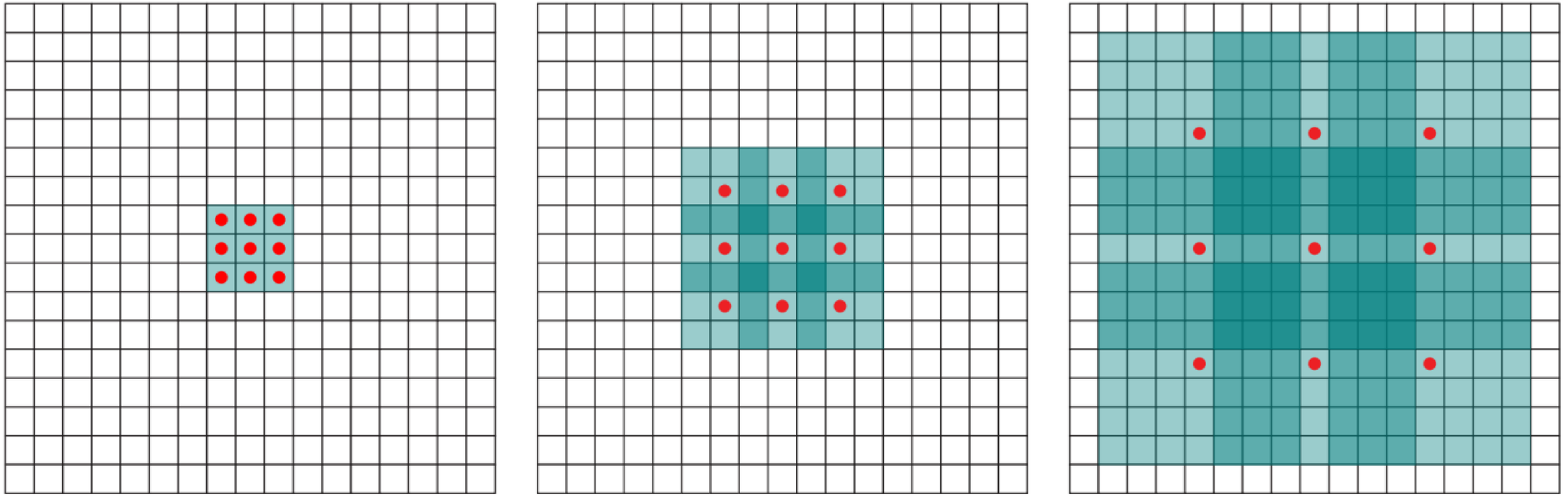


Figure 11: Systematic dilation supports exponential expansion of the receptive field without loss of resolution or coverage. (a) F_1 is produced from F_0 by a 1-dilated convolution; each element in F_1 has a receptive field of 3×3. (b) F_2 is produced from F_1 by a 2-dilated convolution; each element in F_2 has a receptive field of 7×7. (c) F_3 is produced from F_2 by a 4-dilated convolution; each element in F_3 has a receptive field of 15×15. The number of parameters associated with each layer is identical. The receptive field grows exponentially, while the number of parameters grows linearly.

7.6 Conditional random fields (CRFs)

- a trade-off between localization accuracy and classification performance seems to be inherent in CNNs: deeper models with multiple max-pooling layers have proven most successful in classification tasks, however the increased invariance and the large receptive fields of top-level nodes can only yield smooth responses
- as illustrated in Figure 12, CNN score maps can predict the presence and rough position of objects, but cannot really delineate their borders
- previous work has pursued two directions to address this localization challenge
- the first approach is to harness information from multiple layers in the convolutional network in order to better estimate the object boundaries
- the second is to employ a super-pixel representation, essentially delegating the localization task to a low-level segmentation method
- an alternative direction is based on coupling the recognition capacity of CNNs and the fine-grained localization accuracy of fully connected CRFs, which is remarkably successful in addressing the localization challenge, producing accurate semantic segmentation results and recovering object boundaries at a level of detail that is well beyond the reach of existing methods

7.6 Conditional random fields (CRFs)

- traditionally, conditional random fields (CRFs) have been employed to smooth noisy segmentation maps
- typically these models couple neighboring nodes, favoring same-label assignments to spatially proximal pixels
- qualitatively, the primary function of these short-range CRFs is to clean up the spurious predictions of weak classifiers built on top of local hand-engineered features
- compared to these weaker classifiers, modern CNN architectures produce score maps and semantic label predictions which are qualitatively different
- as illustrated in Figure 12, the score maps are typically quite smooth and produce homogeneous classification results
- in this regime, using short-range CRFs can be detrimental, as our goal should be to recover detailed local structure rather than further smooth it
- using contrast-sensitive potentials in conjunction to local-range CRFs can potentially improve localization, but still miss thin structures and typically requires solving an expensive discrete optimization problem

7.6 Conditional random fields (CRFs)

- to overcome these limitations of short-range CRFs, the fully connected CRF model is used
- the model employs the energy function

$$E(\boldsymbol{x}) = \sum_i \theta_i(x_i) + \sum_{ij} \theta_{ij}(x_i, x_j),$$

  where $\boldsymbol{x}$ is the label assignment for pixels
- we use as unary potential $\theta_i(x_i) = -\log P(x_i)$, where $P(x_i)$ is the label assignment probability at pixel i as computed by a CNN
- the pairwise potential has a form that allows for efficient inference while using a fully connected graph, i.e., when connecting all pairs of image pixels, i, j
- in particular, we use the following expression:

$$\theta_{ij}(x_i, x_j) = \mu(x_i, x_j) \left[w_1 \exp\left(-\frac{||p_i - p_j||^2}{2\sigma_\alpha^2} - \frac{||I_i - I_j||^2}{2\sigma_\beta^2} \right) + w_2 \exp\left(-\frac{||p_i - p_j||^2}{2\sigma_\gamma^2} \right) \right],$$

  where $\mu(x_i, x_j) = 1$ if $x_i \neq x_j$, and zero otherwise, which means that only nodes with distinct labels are penalized

7.6 Conditional random fields (CRFs)

- the remaining expression uses two Gaussian kernels in different feature spaces; the first, "bilateral" kernel depends on both pixel positions (denoted as p_i) and RGB color (denoted as I_i), and the second kernel only depends on pixel positions
- the hyperparameters σ_α, σ_β, and σ_γ control the scale of Gaussian kernels
- the first kernel forces pixels with similar color and position to have similar labels, while the second kernel only considers spatial proximity when enforcing smoothness
- crucially, this model is amenable to efficient approximate probabilistic inference
- the message passing updates under a fully decomposable mean field approximation $b(\boldsymbol{x}) = \prod_i b_i(x_i)$ can be expressed as Gaussian convolutions in bilateral space
- high-dimensional filtering algorithms significantly speed-up this computation resulting in an algorithm that is very fast in practice

7.6 Conditional random fields (CRFs)

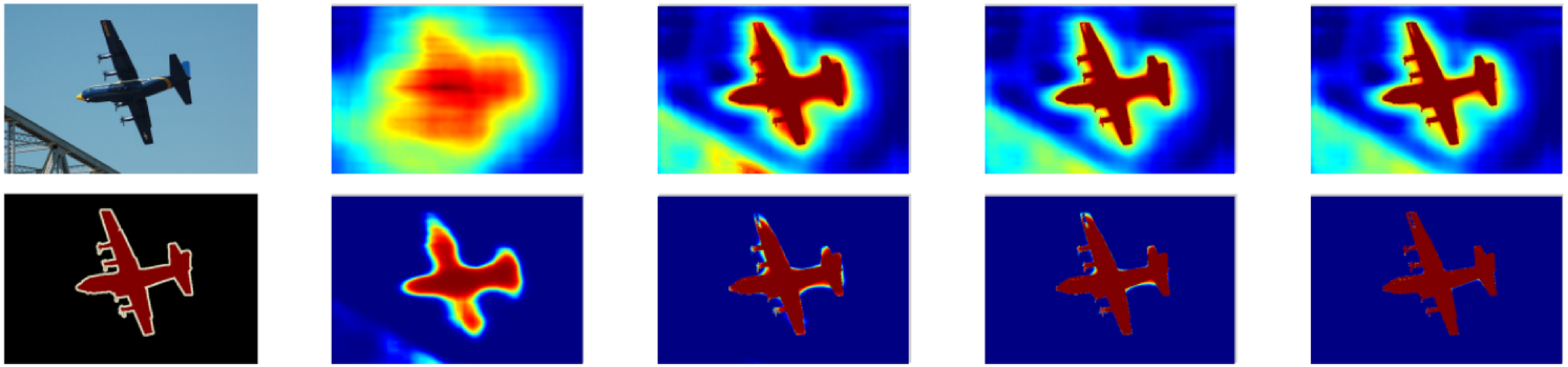


Figure 12: Score map (input before softmax function) and belief map (output of softmax function) for aeroplane. We show the score (1st row) and belief (2nd row) maps after each mean field iteration. The output of the last CNN layer is used as input to the mean field inference.

7.7 DeepLab

- CNNs have shown a remarkable ability to implicitly represent scale, simply by being trained on datasets that contain objects of varying size
- still, explicitly accounting for object scale can improve the CNN's ability to successfully handle both large and small objects
- inspired by the success of the R-CNN spatial pyramid pooling method, which showed that regions of an arbitrary scale can be accurately and efficiently classified by resampling convolutional features extracted at a single scale, DeepLab implemented a variant of that scheme, which uses multiple parallel dilated (atrous) convolutional layers with different sampling rates
- the features extracted for each sampling rate are further processed in separate branches and fused to generate the final result
- the proposed "atrous spatial pyramid pooling" (ASPP) approach is illustrated in Figure 13

7.7 DeepLab

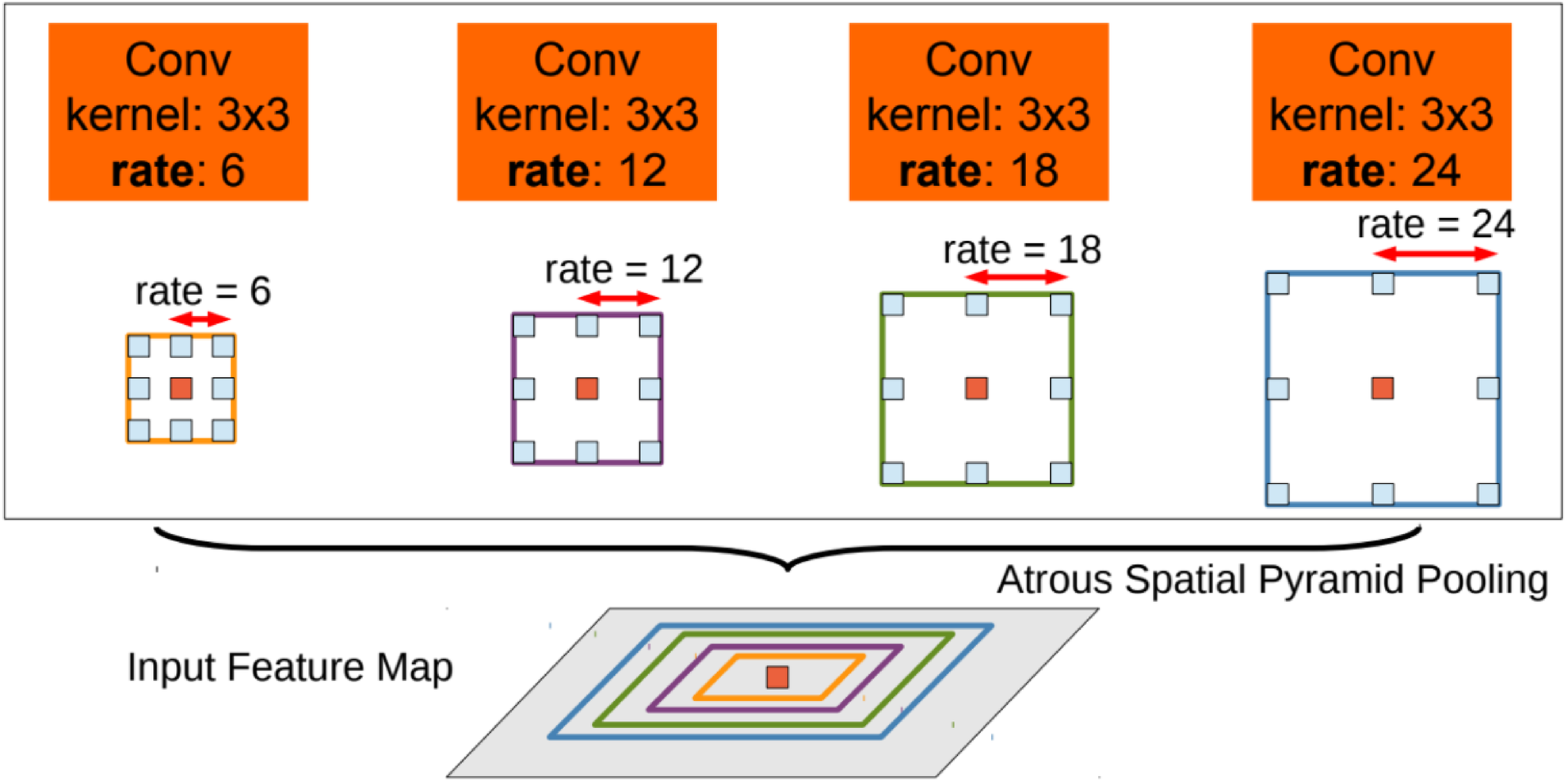


Figure 13: Atrous Spatial Pyramid Pooling (ASPP). To classify the center pixel (orange), ASPP exploits multi-scale features by employing multiple parallel filters with different rates. The effective receptive fields are shown in different colors.

7.7 DeepLab

- a high-level illustration of the DeepLab model is shown in Figure 14
- a deep convolutional neural network (VGG-16 or ResNet-101) trained in the task of image classification is re-purposed to the task of semantic segmentation by:
  1. transforming all the fully connected layers to convolutional layers (i.e., fully convolutional network)
  2. increasing feature resolution through dilated convolutional layers, allowing to compute feature maps every 8 pixels instead of every 32 pixels in the original network
- bilinear interpolation is then employed to upsample by a factor of 8 the score map to reach the original image resolution, yielding the input to a fully connected CRF that refines the segmentation results
- DeepLab can better segment objects at multiple scales via the proposed ASPP

7.7 DeepLab

- the model weights of the ImageNet-pretrained VGG-16 or ResNet-101 networks are fine-tuned to adapt them to the semantic segmentation task in a straightforward fashion
- the 1000-way ImageNet classifier in the last layer is replaced with a classifier having as many targets as the number of semantic classes of our task (including the background, if applicable)
- in order to quadruple the spatial density of computed feature responses in the VGG-16 or ResNet-101 networks, the last two pooling or convolutional layers that decrease resolution have their stride set to 1 to avoid signal decimation, and all subsequent convolutional layers are replaced with dilated convolutional layers having rate $r = 2$
- the loss function is the sum of cross-entropy terms for each spatial position in the CNN output map (subsampled by 8 compared to the original image)
- all positions and labels are equally weighted in the overall loss function (except for unlabeled pixels which are ignored)
- the targets are the ground truth labels (subsampled by 8)
- the CNN and CRF training stages are decoupled, assuming the CNN unary terms are fixed when setting the CRF parameters

7.7 DeepLab

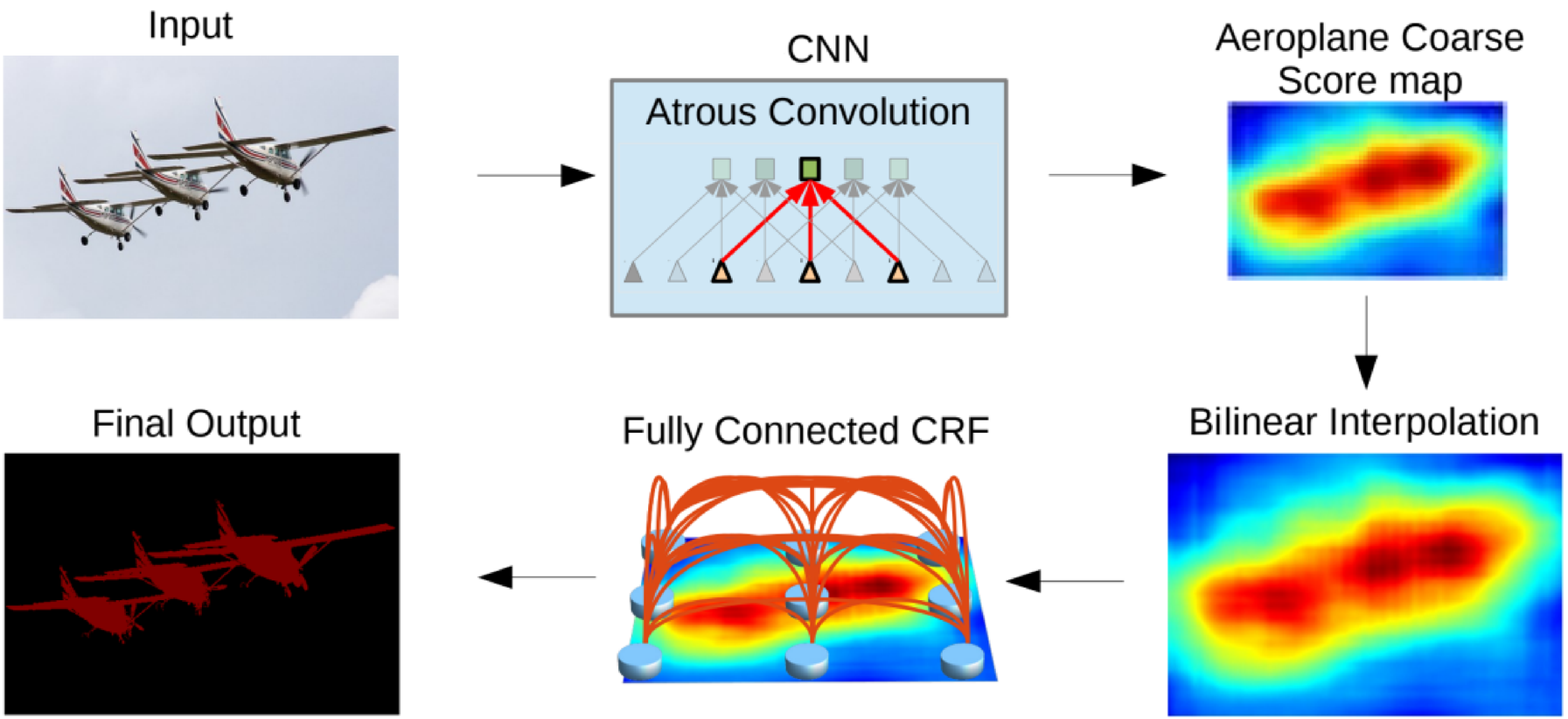
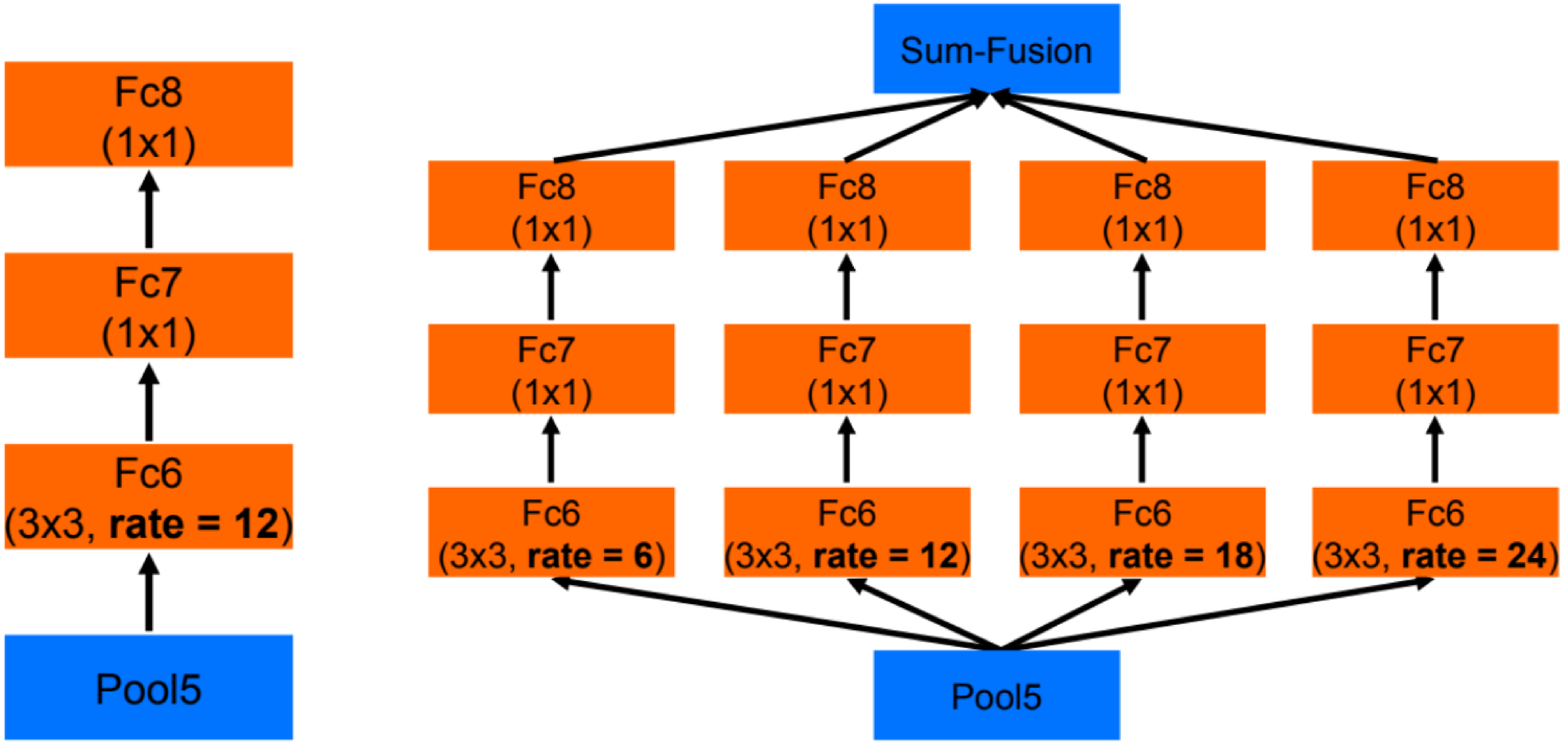


Figure 14: DeepLab model illustration. A deep convolutional neural network such as VGG-16 or ResNet-101 is employed in a fully convolutional fashion, using dilated convolutions to reduce the degree of signal downsampling (from 32$\times$ down to 8$\times$). A bilinear interpolation stage enlarges the feature maps to the original image resolution. A fully connected CRF is then applied to refine the segmentation result and better capture the object boundaries.

7.7 DeepLab

- as shown in Figure 15, ASPP for VGG-16 employs several parallel fc6-fc7-fc8 branches
- they all use 3×3 kernels, but different dilation rates r in the fc6 layer in order to capture objects of different size
- Figure 7 shows two settings:
  - the LargeFOV model, having a single branch with dilation rate $r = 12$
  - the ASPP model, with four branches and dilation rates $r = \{6, 12, 18, 24\}$

7.7 DeepLab



(a) DeepLab-LargeFOV (b) DeepLab-ASPP

Figure 15: DeepLab-ASPP employs multiple filters with different rates to capture objects and context at multiple scales.

7.8 DeepLabv3

- denote by *output_stride* the ratio of input image spatial resolution to final output resolution
- for the CNNs deployed for the task of image classification, the final feature maps (before fully connected layers or global pooling) are 32 times smaller than the input image dimension, and thus *output_stride*$=32$
- if one would like to quadruple the spatial density of computed feature responses in the CNNs (i.e., *output_stride*$=8$), the strides of the last two pooling or convolutional layers that decrease resolution are set to 1 to avoid signal decimation
- then, all subsequent convolutional layers are replaced with dilated convolutional layers having rate $r = 2$

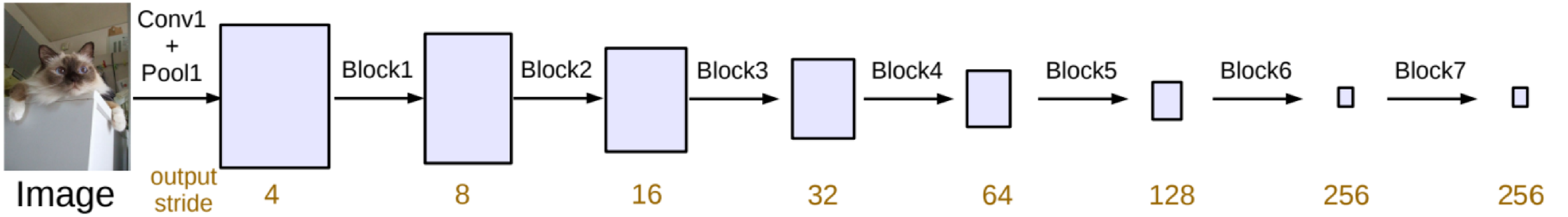
7.8 DeepLabv3

- first, modules with dilated convolution laid out in cascade are designed in the DeepLabv3 model
- to be concrete, several copies of the last ResNet block, denoted as block4 in Figure 16, are duplicated and arranged in cascade
- there are three convolutions in those blocks, and the last convolution contains stride 2 except the one in the last block, similar to the original ResNet
- the motivation behind this model is that the introduced striding makes it easy to capture long range information in the deeper blocks
- for example, the whole image feature could be summarized in the last small resolution feature map, as illustrated in Figure 16(a)
- however, it was discovered that the consecutive striding is harmful for semantic segmentation since detail information is decimated

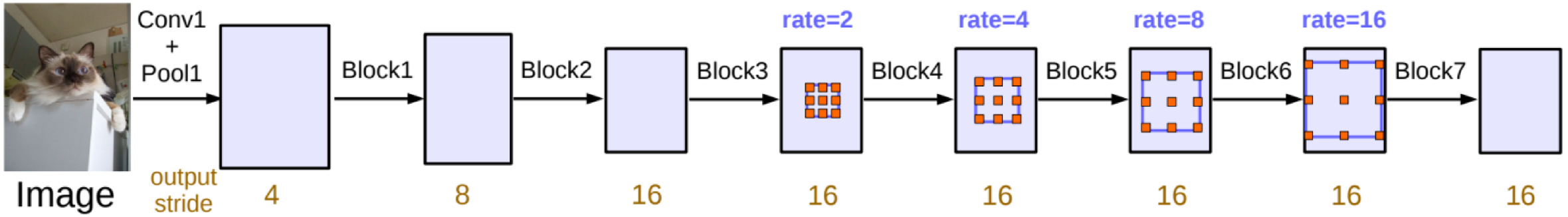
7.8 DeepLabv3

- thus, dilated convolution with rates determined by the desired output stride value are applied, as shown in Figure 16(b), where *output_stride*$=16$
- in this proposed model, cascaded ResNet blocks up to block7 (i.e., extra block5, block6, block7 as replicas of block4) are used, which has *output_stride*$=256$ if no dilated convolution is applied
- motivated by multi-grid methods which employ a hierarchy of grids of different sizes, different dilation rates within block4 to block7 in the DeepLabv3 model are adopted
- in particular, we define as *Multi_Grid*$=(r_1, r_2, r_3)$ the unit rates for the three convolutional layers within block4 to block7
- the final dilation rate for the convolutional layer is equal to the multiplication of the unit rate and the corresponding block rate
- for example, when *output_stride*$=16$ and *Multi_Grid*$=(1,2,4)$, the three convolutions will have *rates* $= 2 \cdot (1,2,4) = (2,4,8)$ in block4, respectively

7.8 DeepLabv3



(a) Going deeper without atrous convolution.



(b) Going deeper with atrous convolution. Atrous convolution with $rate > 1$ is applied after block3 when $output_stride = 16$.

Figure 16: Cascaded modules without and with dilated convolution.

7.8 DeepLabv3

- we revisit the Atrous Spatial Pyramid Pooling (ASPP) proposed in DeepLab, where four parallel dilated convolutions with different dilation rates are applied on top of the feature map
- ASPP is inspired by the success of spatial pyramid pooling, which showed that it is effective to resample features at different scales for accurately and efficiently classifying regions of an arbitrary scale
- different from DeepLab, batch normalization is included within ASPP
- ASPP with different dilation rates effectively captures multi-scale information
- however, it was discovered that, as the sampling rate becomes larger, the number of valid filter weights (i.e., the weights that are applied to the valid feature region, instead of padded zeros) becomes smaller

7.8 DeepLabv3

- to overcome this problem and incorporate global context information to the model, DeepLabv3 adopts image-level features
- specifically, it applies global average pooling on the last feature map of the model, feed the resulting image-level features to a 1×1 convolution with 256 filters (and batch normalization), and then bilinearly upsample the feature to the desired spatial dimension
- in the end, the improved ASPP consists of:
  1. one 1×1 convolution and three 3×3 convolutions with *rates* $= (6, 12, 18)$ when *output_stride*$= 16$ (all with 256 filters and batch normalization), and
  2. the image-level features, as shown in Figure 17
- the resulting features from all the branches are then concatenated and pass through another 1×1 convolution (also with 256 filters and batch normalization) before the final 1×1 convolution which generates the final logits

7.8 DeepLabv3

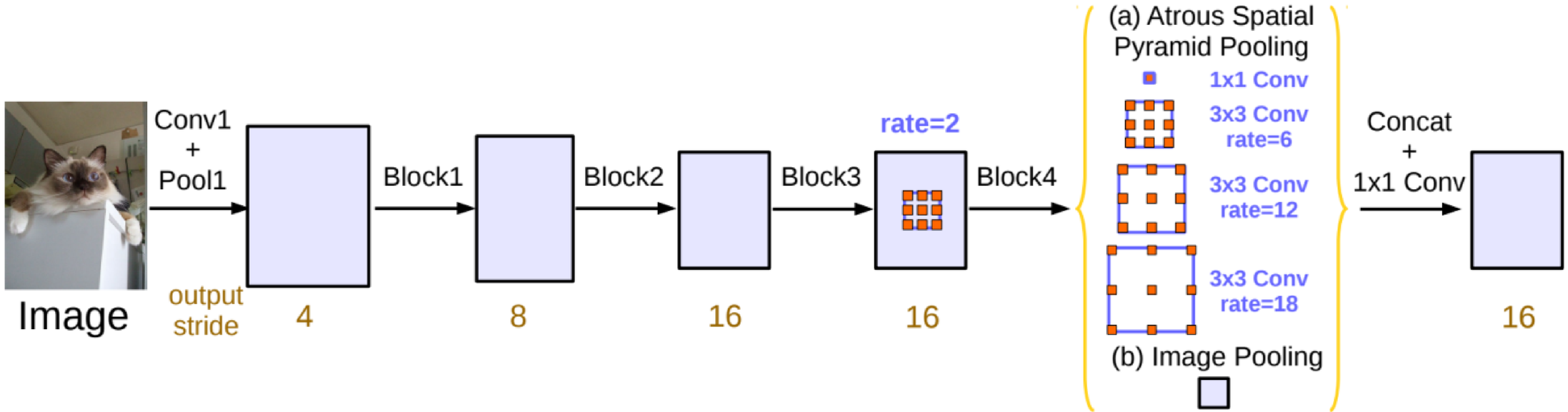


Figure 17: Parallel modules with dilated convolution (ASPP), augmented with image-level features.

Vă mulţumesc!